Луцький національний технічний університет

ЗВІТ

модуль № 1

«Архітектура під змінами»

Виконала: студентка групи ІПЗ-33, **Хвищук Ріта**

Метою роботи є аналіз архітектури системи OrderFlow, виявлення проблем, які ускладнюють внесення змін, та розроблення пропозицій щодо її покращення.

OrderFlow – система оформлення замовлень, яка складається з модулів Orders, Inventory, Payments, Notifications, Reporting та Checkout.

Під час виконання роботи досліджено структуру системи, її основні залежності та сценарії майбутніх змін. На основі аналізу запропоновано поступове вдосконалення архітектури без повного переписування застосунку.

1. Аналіз поточної архітектури OrderFlow

OrderFlow побудована за архітектурою модульного моноліту. Усі модулі працюють в одному процесі, використовують спільну базу даних у пам'яті та розгортаються разом.

Система складається з таких модулів:

- Checkout – координує процес оформлення замовлення.
- Orders – створює замовлення та керує їхніми статусами.
- Inventory – перевіряє наявність товарів і резервує їх.
- Payments – здійснює оплату через зовнішнього провайдера FakePay.
- Notifications – надсилає повідомлення покупцям через FakeMail.
- Reporting – формує звіти про продажі та роботу системи.

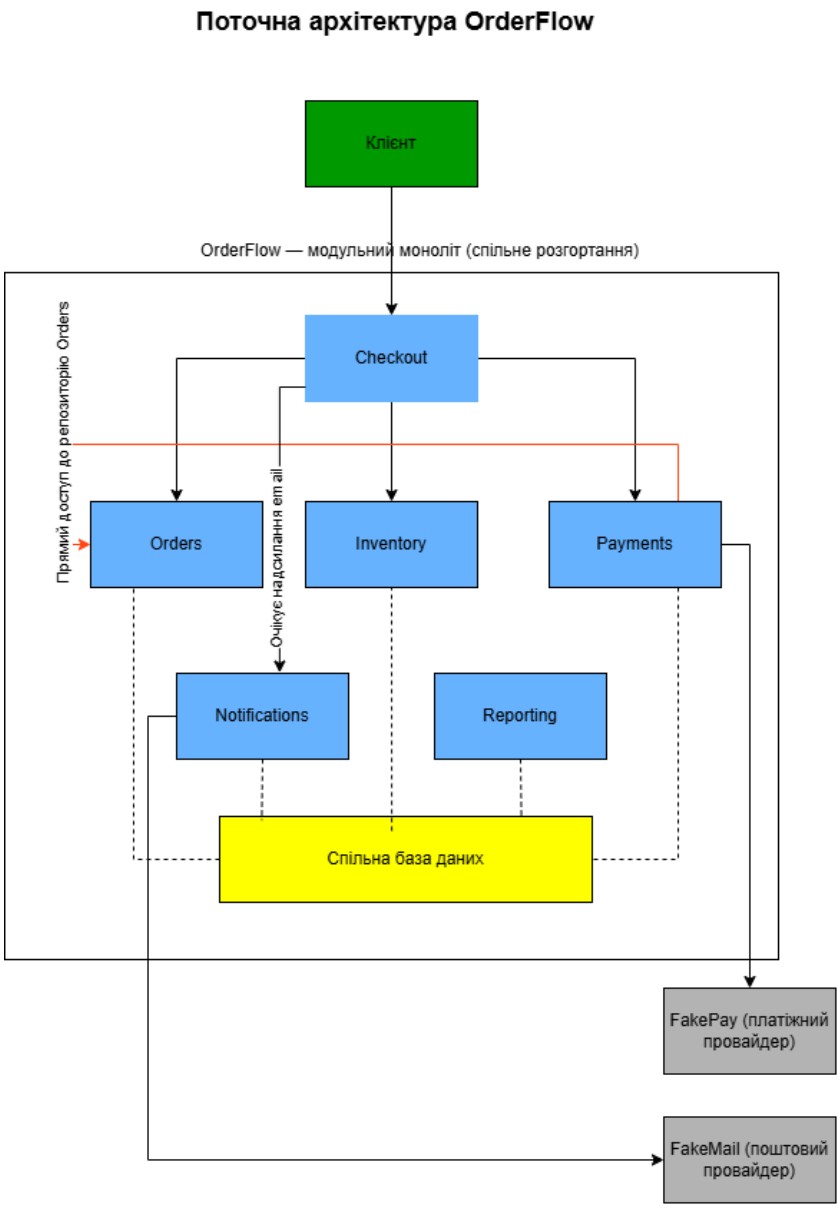
Основні залежності системи:

- Checkout звертається до Orders, Inventory, Payments і Notifications.

- Payments безпосередньо використовує OrderRepository модуля Orders.
- Reporting читає дані інших модулів зі спільної бази даних.
- Checkout очікує завершення надсилання email перед поверненням результату покупцеві.

Через такі залежності зміни в одному модулі можуть впливати на інші частини системи та вимагати додаткового тестування.

Рисунок 1 – Поточна архітектура OrderFlow



2. Запропонована архітектура OrderFlow

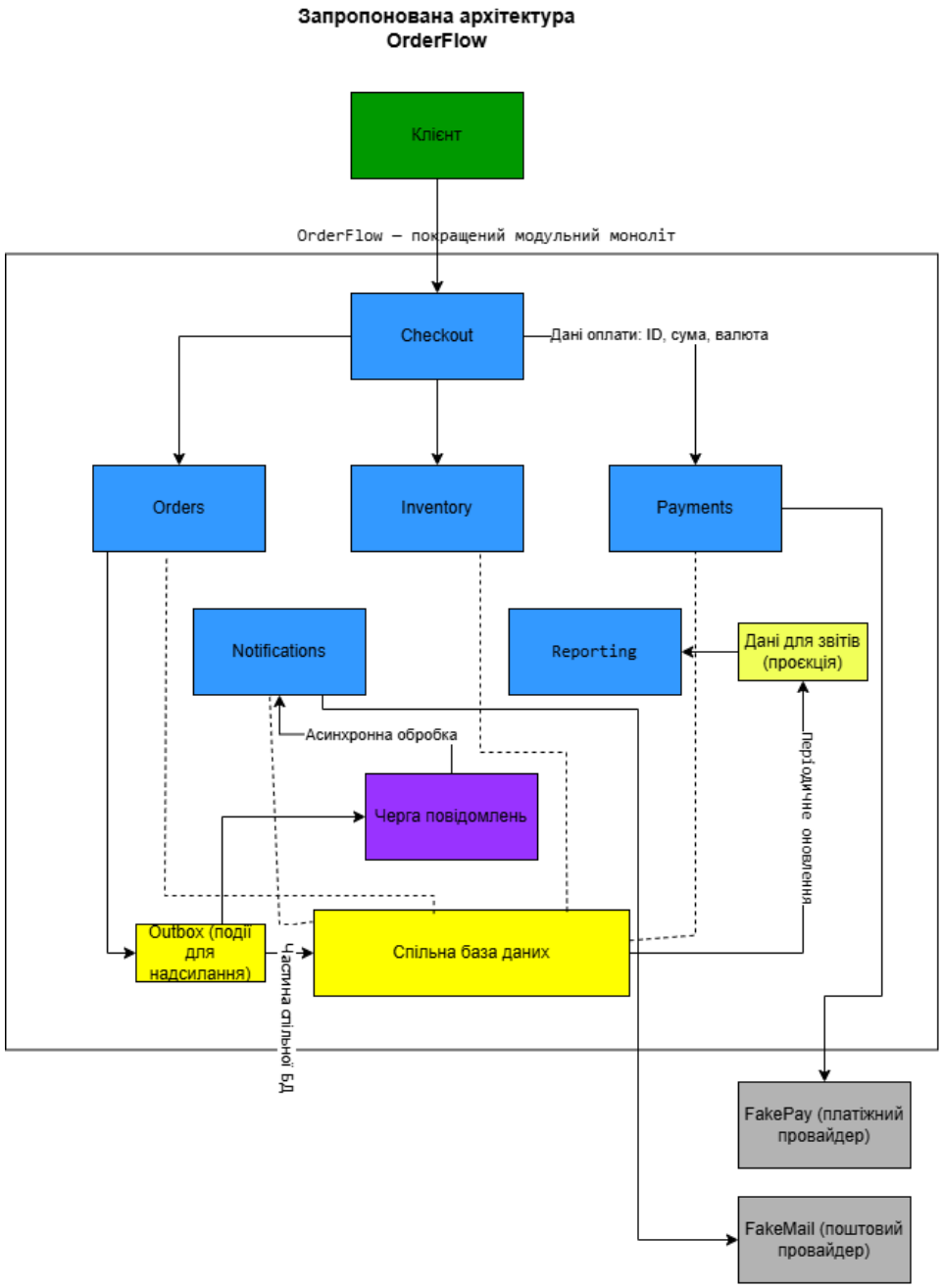
Для покращення системи OrderFlow пропонується поступове вдосконалення модульного моноліту без повного переписування програми.

Основні запропоновані зміни:

1. Асинхронне надсилання повідомлень. Після завершення замовлення подія зберігатиметься в Outbox і передаватиметься до черги повідомлень. Notifications оброблятиме її окремо. Завдяки цьому Checkout не чекатиме надсилання email.
2. Зменшення залежності Payments від Orders. Checkout передаватиме модулю Payments необхідні дані: ідентифікатор замовлення, суму та валюту. Payments більше не звертатиметься безпосередньо до OrderRepository.
3. Оптимізація Reporting. Для формування звітів пропонується використовувати підготовлені дані (проєкцію), які періодично оновлюватимуться. Це зменшить навантаження на основні операції системи.
4. Збереження модульного моноліту. Orders, Inventory, Payments та інші модулі залишатимуться в одному застосунку. Це дозволить покращити архітектуру без значного збільшення складності розгортання та підтримки.
5. Збереження зовнішніх провайдерів. Payments продовжуватиме працювати з FakePay, а Notifications – із FakeMail через відповідні інтерфейси.

Основні виклики запропонованої архітектури – необхідність підтримки черги повідомлень, обробки повторних подій і контролю актуальності даних у звітах.

Рисунок 2 – Запропонована архітектура OrderFlow



3. Обґрунтування архітектурних рішень

Рішення 1. Асинхронне надсилання повідомлень

Проблема: Checkout очікує завершення надсилання email, що збільшує час оформлення замовлення.

Доказ: у файлі CheckoutService.js використовується await notificationService.sendOrderConfirmation(order). Через це затримка поштового провайдера впливає на Checkout.

Рішення: використовувати Outbox і чергу повідомлень для асинхронного надсилання email.

Переваги та недоліки: Checkout не залежатиме від швидкості надсилання листів, але знадобиться обробка черги, повторних спроб і можливих дублікатів.

Як перевірити успіх: вимірювати час оформлення замовлення та частку успішно доставлених повідомлень.

Рішення 2. Усунення прямої залежності Payments від Orders

Проблема: Payments безпосередньо використовує репозиторій замовлень.

Доказ: у PaymentService.js є виклик orderRepository.findById(orderId). За інженерними метриками 48% змін функціональності одночасно зачіпають Orders і Payments.

Рішення: передавати Payments необхідні дані через визначений інтерфейс, не відкриваючи внутрішній репозиторій Orders.

Переваги та недоліки: модулі стануть менш залежними, але потрібно підтримувати узгоджений формат даних і перевіряти правильність суми оплати.

Як перевірити успіх: відстежувати частку змін, які одночасно зачіпають Orders і Payments.

Рішення 3. Оптимізація Reporting

Проблема: Reporting читає операційні дані безпосередньо зі спільної бази, що створює ризик уповільнення системи при зростанні навантаження.

Доказ: у ReportingService.js звіти формуються шляхом перегляду колекцій замовлень, платежів та повідомлень. За бізнес-сценарієм навантаження на звітність може зрости у 8–10 разів.

Рішення: створити підготовлені дані для звітів, які оновлюватимуться окремо від основного процесу оформлення замовлень.

Переваги та недоліки: зменшиться навантаження на операційні дані, але звіти можуть відображати інформацію із затримкою.

Як перевірити успіх: вимірювати час формування звітів, затримку оновлення даних і швидкість роботи Checkout під навантаженням.

Рішення 4. Збереження модульного моноліту

Проблема: часті зміни в різних модулях ускладнюють спільне тестування та розгортання системи.

Доказ: частота релізів знизилася з 5 до 2 на тиждень, а середній час внесення змін зріс із 2 до 6 днів.

Рішення: поступово покращувати межі модулів, не розділяючи одразу всю систему на мікросервіси.

Переваги та недоліки: зберігається простота розгортання й підтримки, але модулі поки не отримують незалежних релізів.

Як перевірити успіх: контролювати частоту релізів, час внесення змін і кількість модулів, які доводиться змінювати для однієї функції.

4. Архітектурні принципи

Для вдосконалення архітектури OrderFlow обрано чотири основні принципи.

4.1. Слабка зв'язаність модулів (Loose Coupling)

Кожен модуль повинен якомога менше залежати від внутрішньої реалізації інших модулів.

Застосування: у поточній архітектурі Payments безпосередньо використовує OrderRepository. Пропонується передавати дані замовлення через чітко визначений інтерфейс, щоб зміни в Orders не вимагали змін у Payments.

4.2. Висока згуртованість (High Cohesion)

Кожен модуль повинен об'єднувати логіку, яка належить до однієї предметної області.

Поточний стан: Orders відповідає за замовлення, Inventory – за резервування товарів, Payments – за оплату. У цьому принцип високої згуртованості вже частково дотриманий.

Запропонована зміна: зберегти ці відповідальності, а надсилання email залишити всередині Notifications, щоб Checkout не залежав від виконання цієї операції.

4.3. Проєктування з урахуванням змін (Design for Change)

Архітектура повинна враховувати функції, які часто змінюються, та зменшувати їхній вплив на інші частини системи.

Поточний стан: Checkout синхронно очікує надсилання email. Через це зміни Notifications можуть вимагати додаткового тестування Checkout.

Запропонована зміна: використати Outbox і чергу повідомлень, щоб Notifications обробляв події асинхронно. Це дозволить частіше змінювати шаблони повідомлень без втручання в логіку оформлення замовлень.

4.4. Поступова еволюція архітектури (Evolutionary Architecture)

Архітектуру потрібно вдосконалювати поступово, враховуючи реальні проблеми та можливості команди.

Застосування: замість негайного переходу до мікросервісів пропонується зберегти модульний моноліт, покращити межі між модулями, додати асинхронні повідомлення та оптимізувати Reporting.

Такий підхід дозволяє зменшити ризики й не зупиняти розробку нових функцій.

5. Аналіз архітектурних ризиків

Для оцінювання ризиків використовується формула:

Очікувана вартість ризику = Ймовірність виникнення × Фінансовий вплив.

Оцінки наведено для періоду одного місяця. Всі суми умовні та використовуються для порівняння ризиків.

Ризик 1. Недоступність платіжного провайдера

Опис: затримки або помилки зовнішнього платіжного провайдера можуть призвести до невдалого оформлення замовлень і втрати продажів.

- Ймовірність: 30% протягом місяця.
- Фінансовий вплив: 20 000 грн.
- Очікувана вартість: 0,30 × 20 000 = 6 000 грн/місяць.
- Зменшення ризику: тайм-аути, контроль повторних платежів, моніторинг затримок і поступове додавання резервного платіжного провайдера.

Ризик 2. Втрата або повторне надсилання повідомлень

Опис: під час переходу на асинхронну обробку можливі втрата подій, повторне надсилання email або затримки доставки.

- Ймовірність: 20% протягом місяця.
- Фінансовий вплив: 5 000 грн.
- Очікувана вартість: 0,20 × 5 000 = 1 000 грн/місяць.
- Зменшення ризику: використання Outbox, повторні спроби доставки, унікальні ідентифікатори повідомлень і контроль черги.

Ризик 3. Застарілі дані у звітах

Опис: підготовлені дані для Reporting можуть оновлюватися із затримкою, через що фінансові звіти не завжди відображатимуть найновіші операції.

- Ймовірність: 40% протягом місяця.
- Фінансовий вплив: 4 000 грн.
- Очікувана вартість: 0,40 × 4 000 = 1 600 грн/місяць.
- Зменшення ризику: контроль затримки оновлення, позначення часу останнього оновлення звіту та можливість отримання актуальних даних для критичних операцій.

Загальна оцінка ризиків

Сумарна очікувана вартість трьох ризиків:

6 000 + 1 000 + 1 600 = 8 600 грн/місяць.

Наведені значення є припущеннями для архітектурного аналізу, а не фактичними втратами компанії. Після впровадження змін оцінки необхідно уточнити на основі реальної статистики.

6. Метрики та Fitness Function

Для оцінювання ефективності запропонованої архітектури OrderFlow визначено п'ять основних метрик.

6.1. Частота релізів (Release Frequency)

Поточне значення: 2 релізи на тиждень.

Цільове значення: не менше 4 релізів на тиждень.

Призначення: показує, наскільки швидко команда може впроваджувати зміни в систему.

6.2. Час внесення змін (Change Lead Time)

Поточне значення: 6 днів.

Цільове значення: не більше 3 днів.

Призначення: дозволяє оцінити, чи стало простіше розробляти, перевіряти та випускати нові функції.

6.3. Кількість модулів, змінених для однієї функції (Modules Changed per Feature)

Поточне значення: 4,3 модуля.

Цільове значення: не більше 2 модулів.

Призначення: показує рівень залежності між модулями. Зменшення цього показника свідчитиме про покращення їхніх меж.

6.4. Частота відкатів (Rollback Rate)

Поточне значення: 9% релізів.

Цільове значення: не більше 4%.

Призначення: допомагає оцінити стабільність системи після впровадження змін.

6.5. Час відновлення після інциденту (MTTR)

Поточне значення: 95 хвилин.

Цільове значення: не більше 45 хвилин.

Призначення: показує, наскільки швидко команда може відновити роботу системи після збою.

6.6. Fitness Function

Fitness Function – це автоматизована перевірка, яка допомагає контролювати дотримання архітектурних правил.

Для OrderFlow пропонується така перевірка:

Правило: модуль Payments не повинен безпосередньо імпортувати або використовувати OrderRepository модуля Orders.

Як працює: під час перевірки змін у репозиторії автоматичний тест аналізує залежності між модулями. Якщо Payments звертається до внутрішнього репозиторію Orders, перевірка завершується помилкою.

Критерій успіху: 0 заборонених залежностей між Payments та OrderRepository.

Ця перевірка допоможе запобігти повторній появі проблеми, яку було виявлено під час аналізу поточної архітектури.

6.7. Оцінювання результатів

Після впровадження архітектурних змін запропоновані метрики необхідно регулярно порівнювати з початковими значеннями.

Поточні показники взято з інженерної документації OrderFlow. Цільові значення є запропонованими орієнтирами, а не вже досягнутими результатами.

7. Оцінювання вартості архітектурних змін

Для порівняння поточної та запропонованої архітектури використовується показник Change Cost – вартість внесення змін.

Формула:

Change Cost = Engineering Hours × Hourly Cost

де:

- Engineering Hours – кількість годин роботи розробників;
- Hourly Cost - вартість однієї години роботи.

Для прикладу приймемо умовну вартість роботи розробника 500 грн/год.

7.1. Поточна архітектура

Розглянемо зміну шаблону повідомлення про замовлення.

У поточній архітектурі Notifications пов'язаний із Checkout, тому зміна може потребувати додаткового регресійного тестування процесу оформлення замовлення.

Припустимо, що розробка й перевірка займають 12 годин.

Change Cost = 12 × 500 = 6 000 грн.

7.2. Запропонована архітектура

Після відокремлення Notifications через асинхронну обробку зміни шаблонів менше впливатимуть на Checkout.

Припустимо, що розробка й перевірка займатимуть 5 годин.

Change Cost = 5 × 500 = 2 500 грн.

7.3. Порівняння результатів

Поточна вартість зміни: 6 000 грн.

Прогнозована вартість після вдосконалення: 2 500 грн.

Очікувана економія на одній зміні:

6 000 − 2 500 = 3 500 грн.

Відносне зменшення вартості:

(3 500 / 6 000) × 100% ≈ 58,3%.

Отже, за прийнятих припущень запропонована архітектура може зменшити вартість типової зміни шаблону повідомлення приблизно на 58%.

Це прогнозний розрахунок. Реальний результат необхідно перевірити після впровадження змін, вимірявши фактично витрачений час.

7.4. Додаткове порівняння вартості змін

Для аналізу розглянуто зміну шаблону повідомлення про замовлення.

1. Modules touched (кількість задіяних модулів)

- Поточна архітектура: безпосередньо змінюється Notifications, але необхідно перевіряти його взаємодію з Checkout.
- Запропонована архітектура: зміни залишаються в Notifications, а Checkout взаємодіє з ним через події.

2. Engineering hours (робочі години)

- Поточна архітектура: умовно 12 годин.
- Запропонована архітектура: умовно 5 годин.

3. Coordination overhead (витрати на узгодження)

- Поточна архітектура: потрібне додаткове узгодження та перевірка впливу змін на Checkout.
- Запропонована архітектура: узгодження переважно обмежується Notifications і форматом події.

4. Testing scope (обсяг тестування)

- Поточна архітектура: тестування Notifications та регресійна перевірка оформлення замовлень.
- Запропонована архітектура: тестування Notifications, обробки подій і перевірка незмінності контракту події.

5. Deployment scope (обсяг розгортання)

- Поточна архітектура: повторне розгортання всього моноліту.

- Запропонована архітектура: повторне розгортання всього моноліту, оскільки незалежне розгортання модулів поки не передбачене.

Висновок: запропонована архітектура зменшує залежність між модулями та обсяг регресійного тестування. Водночас вона не усуває необхідності спільного розгортання, оскільки OrderFlow залишається модульним монолітом. Оцінки робочих годин є умовними.

8. Пріоритети архітектурних характеристик

Для системи OrderFlow визначено такі архітектурні характеристики в порядку важливості.

1. Доступність (Availability) – найвищий пріоритет.

Checkout повинен залишатися працездатним навіть у разі затримок зовнішніх сервісів. Недоступність оформлення замовлень безпосередньо впливає на продажі.

2. Надійність і цілісність даних (Reliability and Data Integrity) – дуже високий пріоритет.

Система повинна запобігати повторним платежам, неправильному резервуванню товарів і втраті інформації про замовлення. Особливо важлива ідемпотентність операцій Inventory.

3. Модифікованість (Modifiability) – високий пріоритет.

Правила оплати та шаблони повідомлень часто змінюються. Архітектура повинна дозволяти оновлювати окремі модулі з мінімальним впливом на інші.

4. Продуктивність і масштабованість (Performance and Scalability) – середній пріоритет.

Reporting повинен витримувати зростання навантаження без значного уповільнення Checkout. Для цього пропонується використовувати окрему проєкцію даних звітності.

Обґрунтування пріоритетів: доступність і правильність оформлення замовлень важливіші за миттєве оновлення звітів або доставку email. Тому повідомлення можна обробляти асинхронно, а звіти – оновлювати з невеликою затримкою.

9. Зв’язок архітектурних рішень із вимогами та сценаріями

Запропоновані архітектурні зміни спрямовані на вирішення конкретних бізнес-вимог і сценаріїв розвитку системи OrderFlow.

Сценарій A. Часті зміни повідомлень

Вимога: шаблони повідомлень змінюються 2–3 рази на тиждень. Ці зміни не повинні впливати на Checkout.

Рішення: відокремити Notifications за допомогою Outbox і асинхронної черги повідомлень. Це дозволить змінювати шаблони без впливу на основний процес оформлення замовлення.

Сценарій B. Додавання другого платіжного провайдера

Вимога: система повинна підтримувати нового платіжного провайдера з іншими помилками та асинхронними підтвердженнями.

Рішення: використовувати спільний інтерфейс платіжного провайдера та відокремити Payments від внутрішньої реалізації Orders. Для асинхронних підтверджень передбачити окрему обробку статусів платежів.

Сценарій C. Зростання навантаження на Reporting

Вимога: навантаження на звітність може зрости у 8–10 разів протягом року. Допускається затримка оновлення звітів на декілька хвилин.

Рішення: використовувати окрему проєкцію даних для Reporting, яка оновлюватиметься періодично та зменшуватиме навантаження на основні операції.

Сценарій D. Надійне резервування товарів

Вимога: повторні запити не повинні призводити до подвійного резервування або продажу товарів понад доступний залишок.

Рішення: зберегти ідемпотентність операцій Inventory та передбачити атомарне оновлення залишків і надійне збереження ключів операцій.

Сценарій E. Доступність Checkout

Вимога: оформлення замовлень повинно бути стійким до затримок і збоїв зовнішніх провайдерів. Надсилання email може виконуватися пізніше.

Рішення: винести повідомлення з основного процесу Checkout. Для Payments передбачити тайм-аути, контроль невизначених результатів оплати та безпечну обробку повторних запитів.

Сценарій F. Знижки залежно від способу оплати та товару

Вимога: система повинна правильно обчислювати знижки до проведення платежу, а результати мають відображатися у звітності.

Рішення: передбачити окрему логіку розрахунку знижок перед викликом Payments. Остаточна сума повинна передаватися на оплату та зберігатися разом із даними замовлення для подальшого використання в Reporting.

Висновок

Запропонована архітектура враховує всі шість сценаріїв розвитку OrderFlow. Основна увага приділяється доступності Checkout, правильності платежів і резервування товарів, зменшенню залежностей між модулями та можливості поступового розвитку системи без повного переписування.